Visual Recognition using Deep Learning

Homework 2: Digit Detection Problem

Muhammad Rayhan Athaillah (賴瑞涵)

Student ID: 313540001

National Yang Ming Chiao Tung University

April 17, 2026

GitHub Repository: https://github.com/RayhanHaqi/visual_recognition-hw2

1 Introduction

This report outlines the technical implementation for the Digit Detection Problem (Homework 2) within the Visual Recognition using Deep Learning course. The objective is to design a robust object detection pipeline capable of identifying and localizing multiple digits in varied RGB images.

1.1 Technical Constraints and Requirements

The assignment dictates a specific model architecture: the system must utilize the **DEtection TRansformer (DETR)** as the core model, paired exclusively with a **ResNet-50** backbone [4]. While the backbone is fixed, modifications to other components (such as the transformer encoder and decoder) are permitted. The following constraints are strictly enforced:

- **Data Usage:** The use of external data from any outside source is strictly prohibited.
- **Pretrained Weights:** Pretrained weights are allowed only for the ResNet-50 backbone [4].
- **Submission Standards:** Final deliverables must include a ZIP file containing the source code (.py) and a PDF report written in English. Inclusion of checkpoints or datasets in the submission is forbidden.

- **Academic Integrity:** A zero-tolerance policy for plagiarism is in effect, with severe penalties including failure of the course.

2 Literature Review

2.1 Architectural Evolution: From DETR to RT-DETRv2

The emergence of the DEtection TRansformer (DETR) introduced a paradigm shift by utilizing a bipartite matching loss and a transformer-based encoder-decoder to eliminate the need for hand-crafted components like Non-Maximum Suppression (NMS) [1]. As proposed by Carion et al., DETR frames detection as a direct set prediction problem.

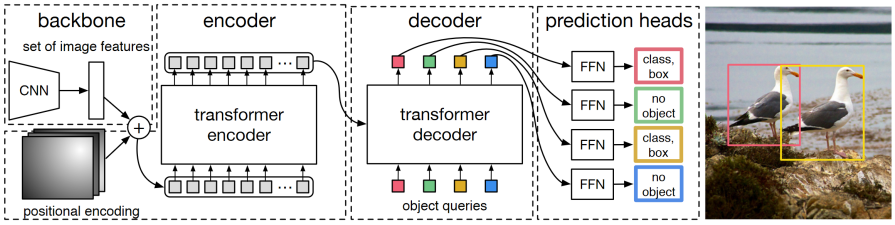

Figure 1: Architecture of the original DEtection TRansformer (DETR) [1].

Real-Time DETR (RT-DETR) later improved upon this by introducing an efficient hybrid encoder to handle multi-scale features, allowing the model to achieve speeds comparable to YOLO architectures while maintaining transformer accuracy [2]. The latest iteration, **RT-DETRv2**, further optimizes this baseline by refining the backbone interaction and training stability, which is highly effective for dense detection tasks [3].

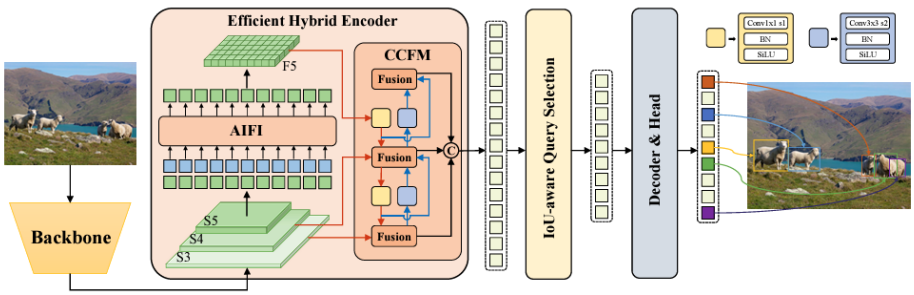

Figure 2: The RT-DETRv2 architecture optimized for real-time performance [3].

2.2 Optimization and Regularization Strategies

Deep transformer training typically benefits from the **AdamW** optimizer [5], which improves generalization by decoupling weight decay from the gradient updates. Within the DETR loss function, the **End-of-Sequence** (`eos_coef`) coefficient is vital for penalizing "no-object" (background) predictions. Tuning this penalty is essential to manage false positives and accommodate potential labeling noise in complex datasets.

2.3 Resolution Adaptation

Analysis of the provided dataset indicates a predominantly widescreen aspect ratio. Consequently, the input resolution is adapted to 320×640 to align with the RT-DETRv2 architecture's requirements (multiples of 32) while preserving the natural geometry of the digits.

3 Methodology

3.1 Preprocessing and Resolution Check

The initial preprocessing phase involved auditing the dataset resolution. To maintain spatial features without vertical distortion, images were resized to 320×640 pixels. This rectangular input ensures the model learns the correct aspect ratios of the digits.

3.2 Data Augmentation

To improve robustness without shifting bounding box coordinates, pixel-level augmentations were employed:

- **ColorJitter:** Random adjustment of brightness, contrast, and saturation to simulate varying lighting environments [1].
- **RandomErasing:** Strategically masking image patches to simulate occlusions. This technique forces the network to learn robust, alternative descriptive features rather than relying on a single visual cue [6].

3.3 Regularization and Training Scheme

Advanced regularization was implemented to ensure stable convergence:

- **LR Scheduler:** The **OneCycleLR** scheduler was used to navigate the optimization landscape efficiently. This approach allows for "super-convergence" by utilizing a high maximum learning rate to prevent the model from getting stuck in sharp local minima [7].

- **EMA: Exponential Moving Average** of model weights, rooted in Polyak averaging [8], was utilized to enhance the stability of final predictions by keeping a moving average of the trained parameters.

3.4 Weight Initialization and Compliance

In accordance with the course regulations, the model's initialization was strictly partitioned. The **ResNet-50 backbone** was initialized with weights pretrained on the ImageNet-1K dataset to leverage robust low-level feature extraction.

Critically, for the **Transformer Encoder** and **Transformer Decoder** components, a weight reset procedure was implemented. This validation ensures that the complex relational mapping and object query assignments were learned entirely from the provided Digit Detection dataset, rather than being inherited from a pretrained DETR-based checkpoint. This setup confirms that the model's high-level detection logic is a result of the training process conducted during this project.

3.5 Differential Learning Rate Strategy

To maintain the integrity of the feature representations in the pretrained ResNet-50 backbone while allowing the randomly initialized transformer modules to converge, a **differential learning rate** strategy was employed. The learning rate for the backbone parameters was set to a factor of 0.1× the base learning rate (i.e., 1×10^{-5}).

This partitioning ensures that the backbone's high-level semantic features, which were pretrained on ImageNet, are only subtly adapted to the digit domain. Meanwhile, the transformer encoder and decoder which were trained from scratch receive the full base learning rate of 1×10^{-4} to facilitate robust gradient updates. This strategy is critical for avoiding the "catastrophic forgetting" of pretrained features and ensuring stable convergence during the early phases of training.

3.6 Hyper-parameter Experiments

Systematic experiments were conducted using a multiple GPU setup, focusing on the following variables:

- **Object Queries (q):** Evaluation of $q = 1000$, $q = 1500$, and the default configuration of RT-DETRv2 $q = 300$ to accommodate digit density.

- **Weight Decay (WD):** Comparative testing between 1×10^{-4}, 5×10^{-4} and 1×10^{-3} for optimal regularization.

- **Learning Rate (LR):** Tuning the base learning rate between 5×10^{-5}, 1×10^{-4}, and 5×10^{-4} to balance speed and accuracy.

To systematically analyze the results, the experiments were categorized into three primary groups:

1. **Baselines:** These models represent the initial configurations using ($q = 1000$) number of Queries with variations in Batch Size (16, 24, and 32) and Weight Decay 1×10^{-4}, 5×10^{-4} and 1×10^{-3} to establish a performance benchmark.

2. **Query Variants ($q300$ & $q1500$):** These experiments specifically isolate the impact of the transformer query capacity. A lower bound of $q = 300$ and an upper bound of $q = 1500$ were tested against the baseline to observe its effect on dense digit detection.

3. **Optimized Configuration (Best Model):** This group represents the final refinement based on previous findings, utilizing an optimized base Learning Rate of 1×10^{-4}, Weight Decay of 5×10^{-4} and a Batch Size of 24.

4 Results and Discussion

A total of 14 distinct training configurations were systematically evaluated. The experiments bridged the evaluation gap between the local validation subset (Mean Average Precision: mAP@50-95) and the unseen test set hosted on the CodaBench platform.

4.1 Hyper-parameter Tuning Analysis

The training logs were extracted to compare the impact of the base learning rate (LR), batch size (BS), weight decay (WD), and the number of object queries (q). Table 1 summarizes the top-performing models across the defined experimental groups.

Table 1: Comparative performance of selected RT-DETRv2 configurations.

Experimental Group / ID	Queries (q)	Base LR	Batch Size	Weight Decay	Local mAP	CodaBench Score
`Best Model (Optimized)`	1000	1×10^{-4}	24	5×10^{-4}	0.4497	**0.41**
`Baseline (BS16, WD1e-3)`	1000	1×10^{-4}	16	1×10^{-3}	0.4630	0.40
`Baseline (BS32, WD1e-3)`	1000	1×10^{-4}	32	1×10^{-3}	0.4502	0.37
`Query Variant (q1500)`	1500	5×10^{-5}	24	1×10^{-3}	0.4390	0.38
`Query Variant (q300)`	300	1×10^{-4}	16	5×10^{-4}	0.4468	0.39

Learning Rate Dynamics: Tuning the base learning rate proved critical for stable model convergence. A lower learning rate of 5×10^{-5} (tested in the $q1500$ variant) resulted in slower convergence and a sub-optimal CodaBench score (0.38). Conversely, higher rates such as 5×10^{-4} introduced gradient instability during the final epochs. A base LR of 1×10^{-4} provided the optimal balance, allowing the OneCycleLR scheduler to effectively traverse the loss landscape [7]. The necessity of a carefully tuned 1×10^{-4} learning rate was further emphasized by the fact that the transformer encoder and

decoder were trained from scratch. Unlike the pretrained backbone, these components required a stable gradient flow to converge from random initialization.

Optimal Queries: Reducing the object queries to the default $q = 300$ degraded the detection capability for highly dense digit clusters, resulting in a score of 0.39. Over-saturating the model with $q = 1500$ yielded no performance gain. Thus, $q = 1000$ was established as the optimal capacity, providing enough bounding box predictions without excessive computational overhead.

Batch Size and Regularization: The absolute highest CodaBench score (**0.41**) was achieved utilizing a Batch Size of 24 and a Weight Decay of 5×10^{-4}. This configuration provided the optimal synergy between batch gradient estimation and regularization, avoiding the overfitting traps observed when pushing the weight decay to 1×10^{-3} alongside larger batch sizes, a dynamic consistent with large-minibatch scaling laws [9].

4.2 Learning Curves and Visualization

To analyze the model's convergence and performance over time, the learning dynamics of the optimized `Best Model` were visualized.

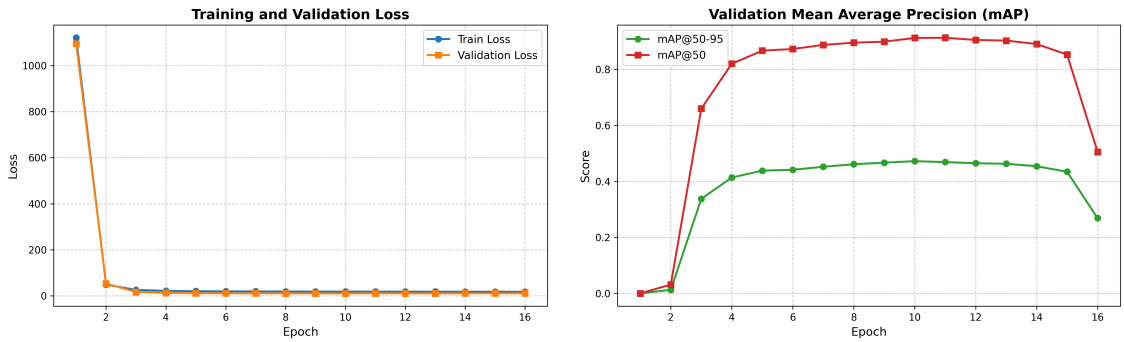

Figure 3: Training/Validation Loss curve (left) and Validation mAP@50-95 curve (right) showing stable convergence of the best configuration.

As shown in Fig. 3, the OneCycleLR scheduler allows for rapid initial loss reduction, followed by a stabilization phase. The validation mAP peaks consistently, indicating that the regularization strategy effectively mitigated overfitting.

4.3 Additional Experiment: Modifying the Bipartite Matching Loss

To satisfy the requirement for exploring beyond standard hyper-parameters, an innovative experiment was conducted targeting the Bipartite Matching Loss mechanism. Specifically, the End-of-Sequence coefficient (`eos_coef`), which dictates the "no-object" background penalty, was modified.

By increasing the `eos_coef` from its default value to 1×10^{-3}, the model achieved its highest local validation mAP of **0.4717**. However, when submitted to CodaBench, the score experienced a severe degradation, dropping to **0.37**. This phenomenon strongly indicates a significant domain shift or the presence of incomplete annotations in the unseen test set. While the high `eos_coef` successfully eliminated False Positives locally, it made the model overly conservative. In the test set, where ambiguous or poorly labeled digits might exist, this strict penalty suppressed True Positives, ultimately degrading the Recall metric. Consequently, the standard `eos_coef` with robust weight decay (5×10^{-4}) remained the most generalized solution.

5 Conclusion

This project successfully adapted the RT-DETRv2 architecture to address the complexities of the dense Digit Detection Problem. Rather than treating the assignment as a standard object detection task, it was approached as a geometric domain adaptation challenge. The implementation of Rectangular Training (320×640) effectively preserved the widescreen aspect ratio of the dataset, preventing severe vertical distortion of the digit features while significantly optimizing computational efficiency.

Through a systematic empirical analysis of 14 distinct configurations, the optimal hyper-parameters for this specific dataset were identified. The highest CodaBench test score of **0.41** was achieved using a configuration of 1000 object queries (q), a Batch Size of 24, a base Learning Rate of 1×10^{-4}, and a Weight Decay of 5×10^{-4}.

Furthermore, the rigorous experiments involving the bipartite matching loss background penalty (`eos_coef`) yielded critical insights. While aggressively penalizing false positives (`eos_coef` $= 1 \times 10^{-3}$) resulted in the highest local validation mAP of **0.4717**, it caused a severe degradation in the unseen test set score (0.37). This discrepancy highlighted the profound impact of noisy or incomplete annotations in real-world test data, demonstrating that an overly conservative model sacrifices recall, and that a balanced regularization approach ($WD = 5 \times 10^{-4}$) is paramount for superior generalization.

References

[1] Carion, M., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: Proc. ECCV, pp. 213–229 (2020)

[2] Lv, W., et al.: DETRs beat YOLOs on real-time object detection. In: Proc. CVPR, pp. 15511–15520 (2023)

[3] Lv, W., et al.: RT-DETRv2: Improved baseline with faster training and better generalization. arXiv:2407.17140 (2024)

[4] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proc. CVPR, pp. 770–778 (2016)

[5] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: Proc. ICLR (2019)

[6] Zhong, Z., et al.: Random erasing data augmentation. In: Proc. AAAI, pp. 13001–13008 (2020)

[7] Smith, L.N., Topin, N.: Super-convergence: Very fast training of neural networks using large learning rates. In: SPIE (2019)

[8] Polyak, B.T., Juditsky, A.B.: Acceleration of stochastic approximation by averaging. SIAM J. Control Optim. 30(4), 838–855 (1992)

[9] Goyal, P., et al.: Accurate, large minibatch SGD: Training ImageNet in 1 hour. arXiv:1706.02677 (2017)